EXPOUNINPAHU 2026 · PROYECTO INTEGRADOR

Especificación de Requisitos de Software (SRS)

ProponeU

Plataforma web de propuestas académicas con inteligencia artificial

---

Integrantes

Sergio Andrés Gutiérrez Marín

Juan Sebastián Gómez Tovar

Docente: Elfar Didier Morantes Sánchez

Programa de Ingeniería de Software — Uninpahu

Entrega 2 — Núcleo funcional · Versión 1.1 · 12/11/2026

Basado en IEEE 830 / ISO/IEC/IEEE 29148

Control de versiones

Versión	Fecha	Autor	Descripción
1.0	05/11/2026	Equipo ProponeU	Versión inicial para la Entrega 1: problema, objetivos, alcance, tecnologías, requisitos, UML inicial, backlog y matriz de trazabilidad.
1.1	12/11/2026	Equipo ProponeU	Entrega 2: diagramas de secuencia (7.3), matriz de trazabilidad con rutas implementadas y estado de implementación (14): modelo de datos, roles, Docker, despliegue y pruebas.

Tabla de contenido

1. Introducción 4
1.1 Propósito 4
1.2 Alcance del producto 4
1.3 Definiciones, acrónimos y abreviaturas 4
1.4 Referencias 4
1.5 Visión general del documento 5
2. Problema a resolver y justificación 6
2.1 ¿Quién tiene el problema? 6
2.2 ¿Cómo se ve actualmente? 6
2.3 ¿Por qué vale la pena resolverlo? 6
2.4 Planteamiento del problema 6
2.5 Justificación 6
3. Objetivos 8
3.1 Objetivo general 8
3.2 Objetivos específicos 8
4. Alcance 9
4.1 El sistema incluye 9
4.2 Fuera del alcance 9
4.3 Alcance por entrega 9
5. Descripción general 10
5.1 Perspectiva del producto 10
5.2 Funciones principales 10
5.3 Actores y características de los usuarios 10
5.4 Restricciones 10
5.5 Suposiciones y dependencias 11
6. Tecnologías 12
7. Arquitectura de la solución 13

7.1 Diagrama de componentes 13
7.2 Diagrama de despliegue 14
7.3 Diagramas de secuencia 14
8. Requisitos funcionales 18
9. Requisitos no funcionales 20
10. Casos de uso 21
10.1 Diagrama de casos de uso 21
10.2 Resumen de casos de uso 21
10.3 Especificación de los casos de uso principales 22
11. Backlog inicial — Historias de usuario 24
12. Matriz de trazabilidad inicial 27
13. Repositorio y organización del proyecto 29
13.1 Estructura del repositorio 29
13.2 Flujo de trabajo (Git Flow) 29
14. Estado de implementación — Entrega 2 30
14.1 Funcionalidades implementadas 30
14.2 Modelo de datos 30
14.3 Roles y permisos 31
14.4 Contenerización con Docker 32
14.5 Despliegue inicial 32
14.6 Pruebas automatizadas 32

Si la tabla aparece vacía, haga clic derecho sobre ella y seleccione «Actualizar campos».

1. Introducción

1.1 Propósito

Este documento especifica los requisitos de ProponeU, plataforma web de propuestas académicas con inteligencia artificial. Está dirigido al docente evaluador, a los jurados de ExpoUninpahu 2026 y al equipo de desarrollo, y sirve como base para el diseño, la implementación, las pruebas y la trazabilidad del proyecto en todas sus entregas.

1.2 Alcance del producto

ProponeU es una aplicación web en la que estudiantes y docentes de Uninpahu proponen actividades académicas complementarias, la comunidad las vota y comenta, y la coordinación académica las revisa y responde. Un microservicio en Node.js integra una API de inteligencia artificial generativa para resumir, clasificar y comparar propuestas, y para generar reportes ejecutivos. El alcance detallado se presenta en la sección 4.

1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
Propuesta	Idea de actividad académica complementaria (taller, charla, semillero, hackatón, salida técnica, club de estudio) registrada en la plataforma.
SRS	Software Requirements Specification — Especificación de Requisitos de Software.
RF / RNF	Requisito funcional / Requisito no funcional.
CU / HU	Caso de uso / Historia de usuario.
IA generativa	Modelos de inteligencia artificial capaces de generar texto a partir de instrucciones (prompts).
Microservicio	Servicio independiente, desplegable por separado, que expone una API para una responsabilidad concreta.
API REST	Interfaz de programación basada en HTTP y JSON.
Swagger / OpenAPI	Especificación estándar y herramienta para documentar y probar APIs REST.
Docker / Docker Compose	Tecnología de contenedores y herramienta para definir y levantar varios contenedores juntos.
ORM	Object-Relational Mapping: capa de Django que traduce modelos de Python a tablas de la base de datos.
Git Flow	Modelo de ramas (main, develop, feature/*, release/*, hotfix/*) para organizar el desarrollo.

1.4 Referencias

- IEEE Std 830-1998, Recommended Practice for Software Requirements Specifications.
- ISO/IEC/IEEE 29148:2018, Systems and software engineering — Life cycle processes — Requirements engineering.

- Guía Institucional del Proyecto Integrador ExpoUninpahu 2026, Programa de Ingeniería de Software.
- Documentación oficial de Django, Node.js, Express, OpenAPI 3, Docker y Render.

1.5 Visión general del documento

La sección 2 describe el problema y su justificación; la 3 los objetivos; la 4 el alcance; la 5 la descripción general del producto, sus actores y restricciones; la 6 las tecnologías; la 7 la arquitectura con los diagramas de componentes y despliegue; las secciones 8 a 10 los requisitos y casos de uso; la 11 el backlog inicial; la 12 la matriz de trazabilidad, y la 13 la organización del repositorio.

2. Problema a resolver y justificación

2.1 ¿Quién tiene el problema?

Estudiantes y docentes de Uninpahu que quieren proponer actividades académicas complementarias (talleres, charlas, semilleros, hackatones, salidas técnicas, clubes de estudio) y los coordinadores académicos y de bienestar que deben decidir cuáles se realizan.

2.2 ¿Cómo se ve actualmente?

- Las ideas circulan por canales informales: grupos de WhatsApp, correos sueltos a coordinación, conversaciones en clase o formatos en papel.
- No existe un registro central: las propuestas se pierden, se repiten o nadie sabe en qué estado están.
- No hay forma objetiva de medir cuánto interés genera una idea en la comunidad; las decisiones dependen de quién insiste más o de percepciones.
- Los coordinadores deben leer, clasificar y resumir a mano cada propuesta, lo que consume tiempo y retrasa las respuestas.
- Quien propone casi nunca recibe retroalimentación sobre si su idea fue aprobada, rechazada o por qué.

2.3 ¿Por qué vale la pena resolverlo?

- Aumenta la participación estudiantil y aprovecha ideas que hoy se pierden.
- Permite decisiones basadas en datos (votos, comentarios, tendencias) y no en percepciones.
- Reduce el trabajo manual de coordinación gracias a resúmenes, clasificación y reportes generados con IA.
- Da transparencia: cada propuesta tiene un estado visible y una respuesta trazable.

2.4 Planteamiento del problema

¿Cómo centralizar, priorizar y dar respuesta trazable a las propuestas de actividades académicas de la comunidad de Uninpahu, reduciendo el trabajo manual de la coordinación mediante inteligencia artificial generativa?

2.5 Justificación

La participación de la comunidad en la definición de actividades académicas fortalece el sentido de pertenencia y la pertinencia de la oferta complementaria. Sin embargo, hoy no existe un canal formal que recoja, organice y priorice esas iniciativas.

ProponeU centraliza el ciclo completo de una propuesta —creación, divulgación, votación, revisión y respuesta— en una aplicación web. La inteligencia artificial generativa aporta valor concreto donde hoy hay trabajo manual: resume y clasifica cada propuesta, detecta ideas repetidas y genera reportes ejecutivos para la coordinación.

Desde el punto de vista académico, el proyecto permite evidenciar las competencias exigidas por el Proyecto Integrador ExpoUninpahu 2026: ingeniería de requisitos, arquitectura basada en

microservicios (Django + Node.js), APIs REST documentadas con Swagger, integración con IA, contenedores Docker y despliegue en la nube.

El equipo ya construyó en clase un prototipo en Django de votación de propuestas. Esa experiencia reduce el riesgo técnico y permite concentrar el esfuerzo en las capacidades nuevas.

3. Objetivos

3.1 Objetivo general

Diseñar, implementar y desplegar una plataforma web basada en Django, integrada con un microservicio Node.js y una API de inteligencia artificial generativa, que permita a la comunidad de Uninpahu proponer, votar y dar seguimiento a actividades académicas, y que apoye a la coordinación en su análisis y aprobación.

3.2 Objetivos específicos

1. Levantar y documentar los requisitos del sistema en un SRS basado en IEEE 830 / ISO/IEC/IEEE 29148, con su matriz de trazabilidad.
2. Modelar la solución con diagramas UML de casos de uso, secuencia, componentes y despliegue que correspondan a la implementación real.
3. Implementar en Django el registro, la autenticación, los roles, el CRUD de propuestas y categorías, la votación, los comentarios y el panel administrativo.
4. Desarrollar un microservicio en Node.js (Express) que exponga una API REST documentada con Swagger/OpenAPI para el análisis con IA y las notificaciones.
5. Integrar una API de IA generativa para resumir y clasificar propuestas, detectar propuestas similares y generar reportes ejecutivos.
6. Contenerizar los servicios con Docker y Docker Compose, y desplegarlos públicamente en Render usando variables de entorno.

4. Alcance

4.1 El sistema incluye

- Registro e inicio de sesión de usuarios con roles: Estudiante, Docente, Coordinador académico y Administrador.
- CRUD de propuestas académicas y de categorías.
- Consulta pública, búsqueda y filtros de propuestas.
- Votación (un voto por usuario por propuesta) y comentarios.
- Flujo de revisión: Borrador → Publicada → En revisión → Aprobada / Rechazada, con observación del coordinador.
- Análisis con IA de cada propuesta: resumen, categoría sugerida, etiquetas y sugerencias de redacción.
- Detección de propuestas similares para evitar duplicados.
- Reporte ejecutivo generado con IA y tablero de estadísticas para la coordinación.
- Notificaciones por correo al cambiar el estado de una propuesta.
- Panel administrativo de Django para gestionar usuarios, roles y catálogos.
- API REST del microservicio documentada públicamente con Swagger.
- Contenedores Docker, orquestación con Docker Compose y despliegue público en Render.

4.2 Fuera del alcance

- Gestión logística de las actividades aprobadas (inscripciones, salones, presupuesto, certificados).
- Integración con los sistemas institucionales reales de Uninpahu (SSO, sistema académico).
- Aplicación móvil nativa (la aplicación web será responsive).
- Pagos o cualquier transacción económica.

4.3 Alcance por entrega

Fecha	Entrega	Resultado esperado
05/11/2026	Entrega 1 — Requisitos y arquitectura	SRS, UML inicial (casos de uso, componentes, despliegue), backlog, repositorio GitHub, matriz de trazabilidad inicial y video.
12/11/2026	Entrega 2 — Núcleo funcional	Registro, login, roles, CRUD de propuestas y categorías, votos, comentarios, revisión, Django Admin, Dockerfile, despliegue inicial y diagrama de secuencia.
19/11/2026	Entrega 3 — Integración	Microservicio Node.js, Swagger público, integración con IA, notificaciones, Docker Compose y UML actualizado.
26/11/2026	ExpoUninpahu — Sustentación	Producto final completo, tablero de estadísticas, documentación final y demostración en vivo.

5. Descripción general

5.1 Perspectiva del producto

ProponeU es un sistema nuevo, inspirado en un prototipo de clase en Django para votar propuestas. Se compone de una aplicación web Django, un microservicio Node.js, una base de datos PostgreSQL y dos servicios externos: una API de IA generativa y un servidor de correo.

5.2 Funciones principales

- Gestión de usuarios, autenticación y roles.
- Gestión de propuestas y categorías (CRUD).
- Participación de la comunidad: búsqueda, votos y comentarios.
- Revisión y respuesta de la coordinación con notificación por correo.
- Análisis con IA: resumen, clasificación, detección de similares y reportes ejecutivos.

5.3 Actores y características de los usuarios

Actor	Descripción	Responsabilidades en el sistema
Visitante	Persona sin sesión iniciada.	Consulta propuestas publicadas, se registra e inicia sesión.
Estudiante	Usuario registrado de la comunidad estudiantil.	Crea, edita y elimina sus propuestas; vota y comenta.
Docente	Usuario registrado del cuerpo docente.	Mismas funciones que el estudiante; sus propuestas se identifican como docentes.
Coordinador académico	Responsable de decidir qué actividades se realizan.	Revisa propuestas, cambia su estado, consulta estadísticas y genera reportes con IA.
Administrador	Responsable técnico de la plataforma.	Gestiona usuarios, roles y categorías desde el panel administrativo.
Microservicio IA	Sistema externo (Node.js + API de IA generativa).	Resume, clasifica, detecta similitudes y genera reportes.
Servicio de correo	Sistema externo (SMTP).	Entrega las notificaciones por correo electrónico.

5.4 Restricciones

- Tecnologías obligatorias definidas por la guía institucional: Django, Node.js, GitHub, Docker, Docker Compose, Swagger/OpenAPI, PostgreSQL, Render/Railway/Vercel y una API de IA generativa.
- Cronograma fijo: Entrega 1 (05/11/2026), Entrega 2 (12/11/2026), Entrega 3 (19/11/2026) y sustentación en ExpoUninpahu (26/11/2026).
- Uso de planes gratuitos: en Render los servicios gratuitos se suspenden por inactividad y la primera petición puede tardar; la API de IA tiene cuotas de uso limitadas.
- Los diagramas UML deben corresponder con la implementación real y actualizarse en cada entrega.

5.5 Suposiciones y dependencias

- Los usuarios cuentan con acceso a internet y un navegador moderno.
- La API de IA generativa estará disponible dentro de su plan gratuito durante el desarrollo y la sustentación.
- Se dispone de una cuenta de correo (SMTP) para el envío de notificaciones.

6. Tecnologías

La selección de tecnologías responde a las tecnologías obligatorias de la guía institucional y a la experiencia previa del equipo con Django.

Capa	Tecnología	Uso en el proyecto
Backend web	Python 3.13 · Django 6	Aplicación principal: vistas, plantillas, autenticación, roles, ORM y panel administrativo.
Frontend	Plantillas Django · Bootstrap 5	Interfaz web responsive en español.
Microservicio	Node.js 20 · Express	API REST para análisis con IA y notificaciones.
Documentación API	Swagger / OpenAPI 3 (swagger-ui-express)	Documentación pública e interactiva de los endpoints en /api/docs.
Inteligencia artificial	API de IA generativa (Google Gemini)	Resúmenes, clasificación, detección de similares y reportes. El proveedor se configura por variable de entorno.
Base de datos	PostgreSQL 16 (SQLite solo en desarrollo)	Persistencia de usuarios, propuestas, votos, comentarios y estados.
Contenedores	Docker · Docker Compose	Un contenedor por servicio (django_app, ms_ia, db) y orquestación local.
Despliegue	Render (Web Services + PostgreSQL gestionado)	Publicación de ambos servicios con despliegue automático desde GitHub.
Control de versiones	Git · GitHub (Git Flow)	Repositorio, ramas main/develop/feature, issues y tablero del backlog.
Calidad	Django TestCase · Jest · flake8 · ESLint	Pruebas automatizadas y estilo de código.

7. Arquitectura de la solución

La solución sigue una arquitectura de microservicios ligera: Django es la aplicación principal y dueña de los datos; Node.js es un microservicio especializado que aísla las integraciones con terceros (IA y correo). Ambos se comunican por REST/JSON y se empaquetan como contenedores Docker.

7.1 Diagrama de componentes

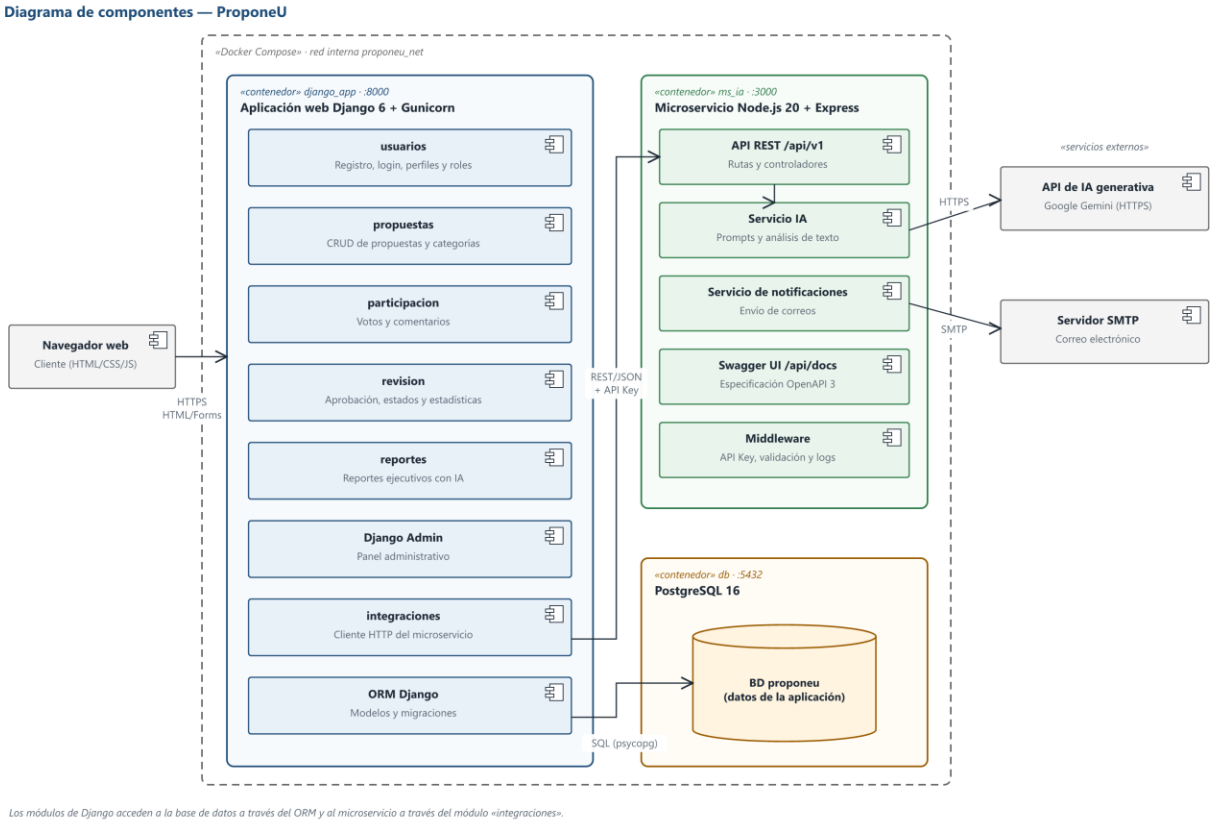


Figura 1. Diagrama de componentes de ProponeU.

Componente	Responsabilidad
django_app (Django 6 + Gunicorn)	Aplicación principal. Sus módulos son: usuarios (registro, login, perfiles y roles), propuestas (CRUD de propuestas y categorías), participacion (votos y comentarios), revision (flujo de aprobación y estadísticas), reportes (reportes con IA), Django Admin (panel administrativo) e integraciones (cliente HTTP que se comunica con el microservicio). Todos acceden a los datos a través del ORM de Django.
ms_ia (Node.js 20 + Express)	Microservicio que concentra lo que depende de terceros. Expone una API REST en /api/v1 protegida con API Key, un servicio de IA que construye los prompts y llama a la API de IA generativa, un servicio de notificaciones que envía correos por SMTP y la documentación Swagger UI en /api/docs.
db (PostgreSQL 16)	Base de datos relacional de la aplicación Django: usuarios, roles, propuestas, categorías, votos, comentarios, historial de estados y resultados del análisis de IA.

Componente	Responsabilidad
API de IA generativa	Servicio externo (Google Gemini) consumido solo por el microservicio. La clave se guarda en la variable GEMINI_API_KEY.
Docker Compose	Levanta los tres contenedores en una red interna (proponeu_net) para el desarrollo local con un solo comando.

7.2 Diagrama de despliegue

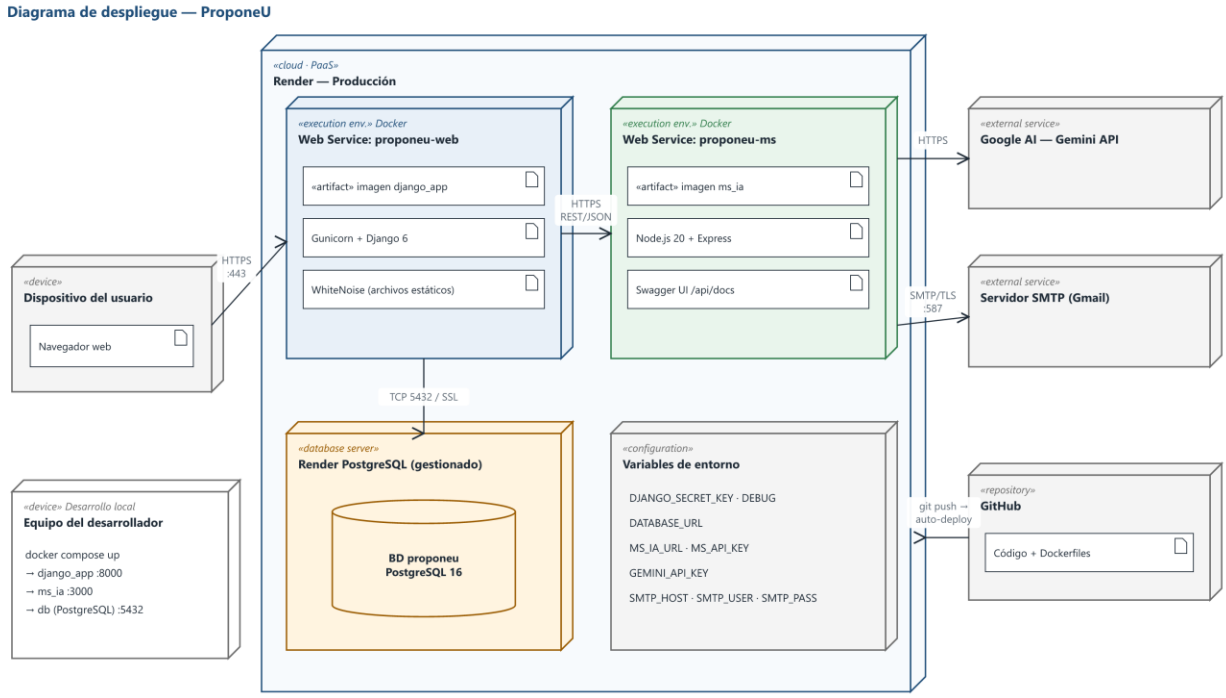


Figura 2. Diagrama de despliegue de ProponeU.

- En producción ambos servicios se publican en Render como Web Services construidos desde sus Dockerfiles. La base de datos es un PostgreSQL gestionado por Render.
- El usuario accede por HTTPS a proponeu-web (Django). Django consulta PostgreSQL por TCP con SSL y llama al microservicio proponeu-ms por HTTPS con REST/JSON y API Key.
- El microservicio es el único que se comunica con la API de Gemini y con el servidor SMTP.
- Los secretos (DJANGO_SECRET_KEY, DATABASE_URL, MS_API_KEY, GEMINI_API_KEY, credenciales SMTP) se configuran como variables de entorno en Render y nunca se suben al repositorio.
- Cada push a la rama main en GitHub dispara el despliegue automático. En el equipo de cada desarrollador, docker compose up levanta el mismo entorno de forma local.

7.3 Diagramas de secuencia

Primera versión de los diagramas de secuencia, elaborada a partir de la implementación de la Entrega 2. Muestran la interacción entre el navegador, las vistas de Django, los formularios, los modelos del ORM y la base de datos en los tres flujos principales del núcleo funcional.

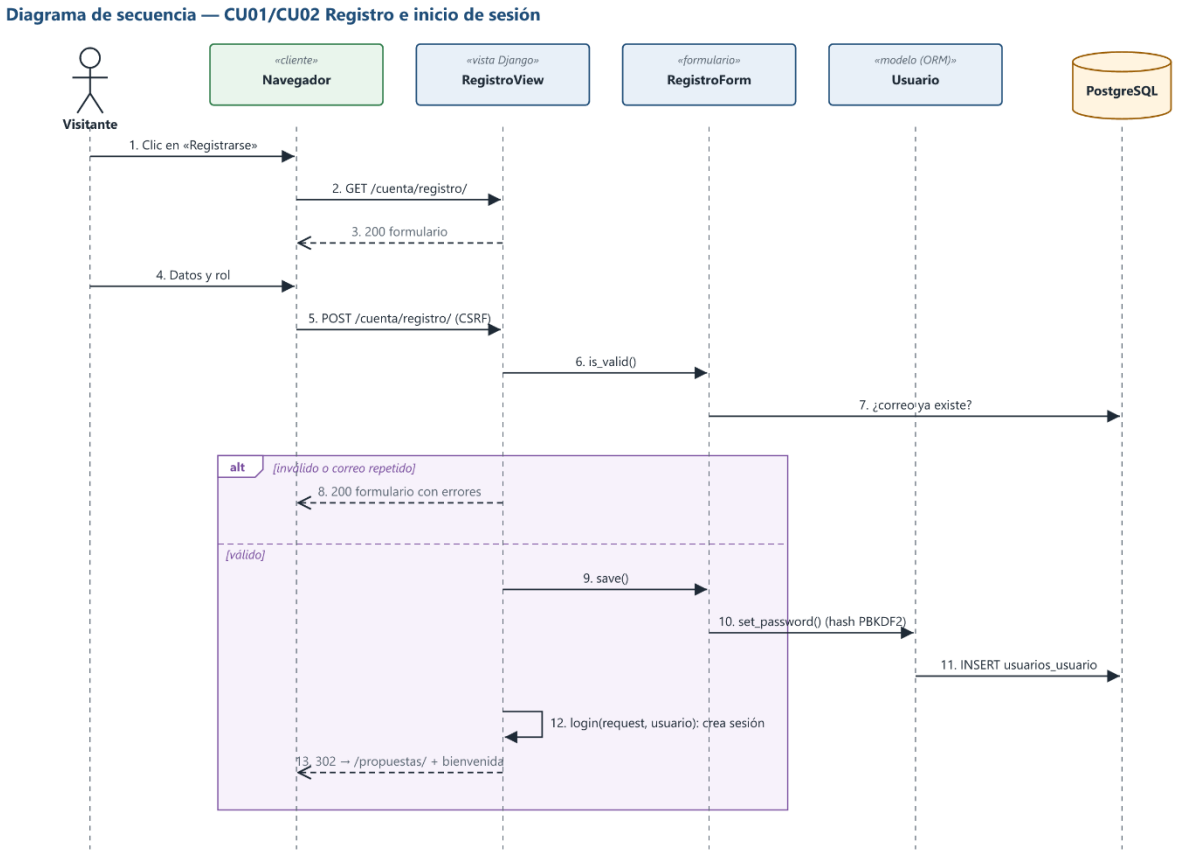


Figura 3. Secuencia de registro e inicio de sesión (CU01, CU02).

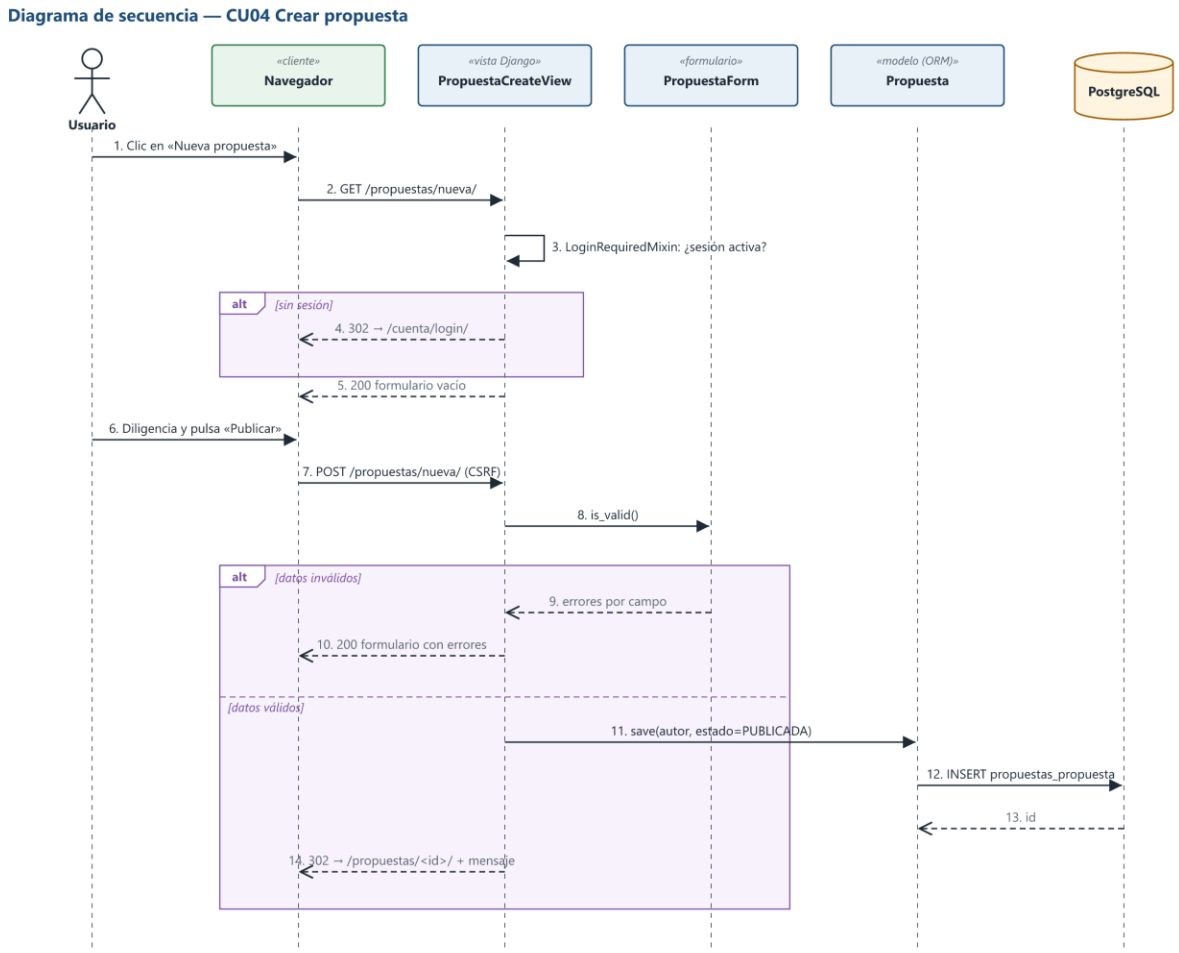


Figura 4. Secuencia de creación de una propuesta (CU04).

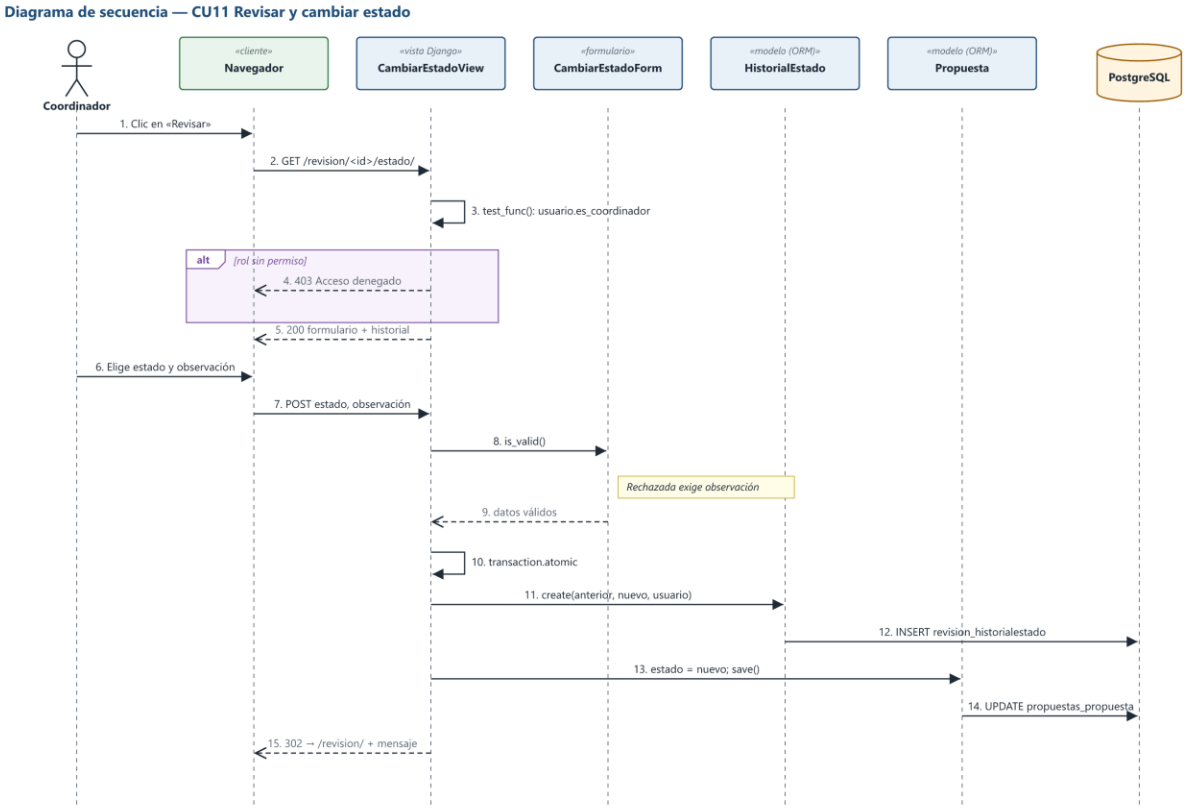


Figura 5. Secuencia de revisión y cambio de estado (CU11).

8. Requisitos funcionales

ID	Requisito	Prioridad	CU	HU
RF01	El sistema debe permitir que un visitante se registre como Estudiante o Docente con nombre, correo, programa y contraseña.	Alta	CU01	HU01
RF02	El sistema debe permitir iniciar y cerrar sesión con correo y contraseña.	Alta	CU02	HU02
RF03	El usuario debe poder consultar y actualizar los datos de su perfil.	Media	CU03	HU03
RF04	El sistema debe restringir cada funcionalidad según el rol: Estudiante, Docente, Coordinador académico o Administrador.	Alta	CU15	HU15
RF05	Un usuario registrado debe poder crear una propuesta con título, descripción, objetivo, público objetivo, modalidad, fecha tentativa y categoría.	Alta	CU04	HU04
RF06	El autor debe poder editar o eliminar su propuesta mientras esté en estado Borrador o Publicada.	Alta	CU05	HU05
RF07	Cualquier persona debe poder listar las propuestas publicadas y buscarlas por texto, categoría, estado y número de votos.	Alta	CU06	HU06
RF08	Un usuario registrado debe poder votar a favor o en contra de una propuesta, con un único voto por propuesta que puede cambiar.	Alta	CU07	HU07
RF09	Un usuario registrado debe poder comentar las propuestas publicadas.	Media	CU08	HU08
RF10	Al crear o editar una propuesta, el sistema debe obtener del microservicio IA un resumen, una categoría sugerida, etiquetas y sugerencias de redacción.	Alta	CU09	HU09
RF11	Al crear una propuesta, el sistema debe mostrar propuestas existentes similares para evitar duplicados.	Media	CU10	HU10
RF12	El Coordinador académico debe poder cambiar el estado de una propuesta (En revisión, Aprobada, Rechazada) registrando una observación.	Alta	CU11	HU11
RF13	El sistema debe notificar por correo al autor cuando su propuesta cambie de estado.	Media	CU16	HU12
RF14	El Coordinador académico debe poder generar con IA un reporte ejecutivo de un periodo con las propuestas más votadas y las tendencias por categoría.	Media	CU12	HU13
RF15	El Coordinador académico debe poder consultar un tablero con estadísticas de propuestas por estado, categoría y votos.	Baja	CU13	HU14
RF16	El Administrador debe poder crear, editar y eliminar categorías.	Media	CU14	HU16
RF17	El Administrador debe poder gestionar usuarios y asignar roles desde el panel administrativo.	Alta	CU15	HU15

ID	Requisito	Prioridad	CU	HU
RF18	El microservicio debe exponer su API REST documentada con Swagger/OpenAPI en una URL pública.	Alta	CU09, CU10, CU12, CU16	HU17

9. Requisitos no funcionales

ID	Categoría	Requisito
RNF01	Seguridad	Las contraseñas se almacenan con hash (PBKDF2 de Django); todos los formularios usan protección CSRF y en producción se exige HTTPS.
RNF02	Autorización	Cada vista valida el rol del usuario; un usuario solo puede modificar o eliminar sus propias propuestas.
RNF03	Configuración	Claves, contraseñas y URLs se leen de variables de entorno. El repositorio solo contiene un archivo .env.example sin secretos.
RNF04	Rendimiento	Las páginas principales responden en menos de 2 segundos con 50 usuarios concurrentes. Las llamadas a la IA tienen un tiempo límite de 15 segundos.
RNF05	Disponibilidad	Si el microservicio o la IA no responden, la aplicación Django sigue funcionando: la propuesta se guarda y el análisis queda pendiente para reintentar.
RNF06	Usabilidad	Interfaz en español, responsive (Bootstrap 5) y usable desde celular y computador.
RNF07	Portabilidad	Cada servicio tiene su Dockerfile y el sistema completo se levanta con docker compose up.
RNF08	Mantenibilidad	Código según PEP 8 y ESLint, ramas con Git Flow y pruebas automatizadas con al menos 60 % de cobertura en la lógica de negocio.
RNF09	Interoperabilidad	La comunicación Django → Node.js usa REST/JSON autenticado con API Key, documentado en OpenAPI 3.
RNF10	Privacidad	A la IA solo se envía el contenido de la propuesta, nunca datos personales.
RNF11	Auditoría	Cada cambio de estado guarda fecha, usuario que lo realizó y observación.

10. Casos de uso

10.1 Diagrama de casos de uso

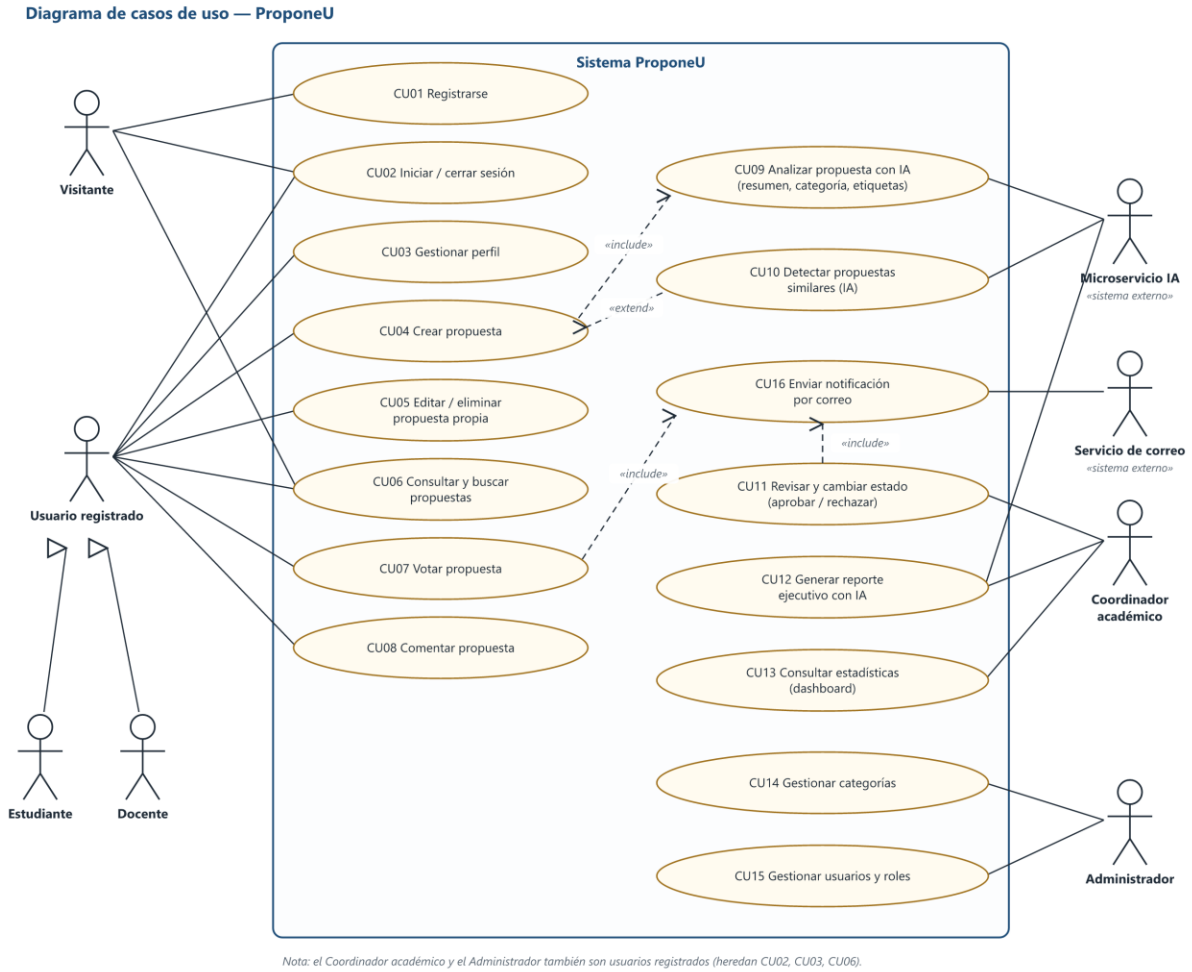


Figura 6. Diagrama de casos de uso de ProponeU.

10.2 Resumen de casos de uso

ID	Caso de uso	Actores	Descripción
CU01	Registrarse	Visitante	El visitante crea su cuenta como Estudiante o Docente.
CU02	Iniciar / cerrar sesión	Visitante, Usuario registrado	Autenticación con correo y contraseña.
CU03	Gestionar perfil	Usuario registrado	Consultar y actualizar los datos personales.
CU04	Crear propuesta	Usuario registrado	Registrar una nueva propuesta académica. Incluye CU09.
CU05	Editar / eliminar propuesta propia	Usuario registrado	Modificar o borrar una propuesta propia en Borrador o Publicada.

ID	Caso de uso	Actores	Descripción
CU06	Consultar y buscar propuestas	Visitante, Usuario registrado	Listado público con búsqueda y filtros.
CU07	Votar propuesta	Usuario registrado	Votar a favor o en contra; un voto por usuario. Incluye CU16 cuando aplica.
CU08	Comentar propuesta	Usuario registrado	Agregar comentarios a una propuesta publicada.
CU09	Analizar propuesta con IA	Microservicio IA	Obtener resumen, categoría sugerida, etiquetas y sugerencias de redacción.
CU10	Detectar propuestas similares	Microservicio IA	Extiende CU04: muestra propuestas parecidas antes de publicar.
CU11	Revisar y cambiar estado	Coordinador académico	Pasar la propuesta a En revisión, Aprobada o Rechazada con observación. Incluye CU16.
CU12	Generar reporte ejecutivo con IA	Coordinador académico, Microservicio IA	Reporte de un periodo con propuestas destacadas y tendencias.
CU13	Consultar estadísticas	Coordinador académico	Tablero con totales por estado, categoría y votos.
CU14	Gestionar categorías	Administrador	CRUD de categorías de propuestas.
CU15	Gestionar usuarios y roles	Administrador	Administrar cuentas y asignar roles en el panel de Django.
CU16	Enviar notificación por correo	Servicio de correo	El microservicio envía el correo al autor de la propuesta.

10.3 Especificación de los casos de uso principales

CU04 — Crear propuesta

Actores	Usuario registrado (principal), Microservicio IA (secundario)
Precondición	El usuario inició sesión.
Flujo principal	1. El usuario selecciona «Nueva propuesta». 2. El sistema muestra el formulario con título, descripción, objetivo, público objetivo, modalidad, fecha tentativa y categoría. 3. El usuario diligencia los datos y selecciona «Analizar con IA». 4. El sistema envía el título y la descripción al microservicio IA (CU09) y muestra el resumen, la categoría sugerida, las etiquetas y las sugerencias de redacción. 5. El sistema muestra las propuestas similares encontradas (CU10). 6. El usuario ajusta el texto si lo desea y selecciona «Publicar» o «Guardar como borrador». 7. El sistema guarda la propuesta con el estado elegido y muestra su detalle.
Flujos alternos	4a. El microservicio no responde en 15 s: el sistema avisa que el análisis no está disponible y permite publicar igual; el análisis queda pendiente. 6a. Faltan campos obligatorios: el sistema marca los campos y no guarda.

Postcondición	La propuesta queda registrada con su autor, fecha y estado.

CU07 — Votar propuesta

Actores	Usuario registrado
Precondición	El usuario inició sesión y la propuesta está en estado Publicada.
Flujo principal	1. El usuario abre el detalle de una propuesta. 2. El usuario selecciona «A favor» o «En contra». 3. El sistema registra el voto (o lo reemplaza si ya existía) y actualiza el conteo.
Flujos alternos	2a. El usuario no ha iniciado sesión: el sistema lo redirige al inicio de sesión. 2b. El usuario es el autor de la propuesta: el sistema no permite votar su propia propuesta.
Postcondición	Existe como máximo un voto por usuario para la propuesta.

CU11 — Revisar y cambiar estado

Actores	Coordinador académico (principal), Servicio de correo (secundario)
Precondición	El usuario inició sesión con rol Coordinador académico.
Flujo principal	1. El coordinador abre la bandeja de propuestas publicadas, ordenadas por votos. 2. El coordinador consulta el detalle, el resumen de la IA, los votos y los comentarios. 3. El coordinador selecciona el nuevo estado (En revisión, Aprobada o Rechazada) y escribe una observación. 4. El sistema guarda el cambio con fecha y usuario (RNF11). 5. El sistema solicita al microservicio el envío del correo al autor (CU16).
Flujos alternos	3a. Rechazo sin observación: el sistema exige la observación. 5a. Falla el envío del correo: el cambio de estado se conserva y el error queda registrado para reintento.
Postcondición	La propuesta tiene un nuevo estado visible para su autor y la comunidad.

11. Backlog inicial — Historias de usuario

Las historias se priorizan con MoSCoW (Must, Should, Could) y se estiman en puntos de historia (escala Fibonacci). Cada historia se registrará como issue en GitHub y se gestionará en el tablero del proyecto.

ID	Historia de usuario	Criterios de aceptación	Prior.	Pts	Entrega
HU01	Como visitante, quiero registrarme como estudiante o docente, para poder proponer, votar y comentar.	• El formulario pide nombre, correo, programa, rol y contraseña. • No se permite un correo ya registrado.	Must	3	Entrega 2
HU02	Como usuario registrado, quiero iniciar y cerrar sesión, para acceder de forma segura a mis funciones.	• Con credenciales válidas entro a la página principal. • Con credenciales inválidas veo un mensaje de error. • Al cerrar sesión no puedo entrar a páginas protegidas.	Must	2	Entrega 2
HU03	Como usuario registrado, quiero editar mi perfil, para mantener mis datos actualizados.	• Puedo cambiar nombre, programa y contraseña. • Los cambios se ven de inmediato.	Could	2	Entrega 2
HU04	Como estudiante o docente, quiero crear una propuesta de actividad académica, para que la comunidad la conozca y la apoye.	• Título, descripción y categoría son obligatorios. • Puedo guardarla como borrador o publicarla. • La propuesta queda asociada a mi usuario.	Must	5	Entrega 2
HU05	Como autor de una propuesta, quiero editar o eliminar mi propuesta, para corregir errores o retirarla.	• Solo veo las opciones en mis propuestas. • No puedo editar una propuesta En revisión, Aprobada o Rechazada. • Eliminar pide confirmación.	Must	3	Entrega 2
HU06	Como miembro de la comunidad, quiero buscar y filtrar propuestas, para encontrar las que me interesan.	• Puedo buscar por texto. • Puedo filtrar por categoría y estado. • Puedo ordenar por más votadas o más recientes.	Must	3	Entrega 2
HU07	Como usuario registrado, quiero votar a favor o en contra de una propuesta, para expresar mi interés.	• Solo tengo un voto por propuesta y puedo cambiarlo. • No puedo votar mi propia propuesta. • El conteo se actualiza al votar.	Must	3	Entrega 2
HU08	Como usuario registrado, quiero comentar una propuesta, para aportar ideas o hacer preguntas.	• El comentario muestra autor y fecha. • No se permiten comentarios vacíos.	Should	2	Entrega 2

ID	Historia de usuario	Criterios de aceptación	Prior.	Pts	Entrega
HU09	Como autor de una propuesta, quiero que la IA resuma mi propuesta, sugiera categoría y etiquetas y me dé sugerencias de redacción, para presentarla de forma clara.	• Al pulsar «Analizar con IA» veo resumen, categoría, etiquetas y sugerencias. • Si la IA falla, puedo publicar igual. • El resumen se guarda con la propuesta.	Must	8	Entrega 3
HU10	Como autor de una propuesta, quiero ver propuestas similares antes de publicar, para no duplicar ideas y unirme a una existente.	• Se muestran hasta 3 propuestas similares con enlace. • Si no hay similares, se informa.	Should	5	Entrega 3
HU11	Como coordinador académico, quiero cambiar el estado de una propuesta con una observación, para dar respuesta formal a la comunidad.	• Puedo elegir En revisión, Aprobada o Rechazada. • La observación es obligatoria al rechazar. • Queda registro de fecha y usuario.	Must	5	Entrega 2
HU12	Como autor de una propuesta, quiero recibir un correo cuando cambie el estado de mi propuesta, para enterarme sin entrar a la plataforma.	• El correo incluye título, nuevo estado y observación. • Si el correo falla, el cambio de estado no se pierde.	Should	3	Entrega 3
HU13	Como coordinador académico, quiero generar con IA un reporte ejecutivo de un periodo, para tomar decisiones rápidas basadas en datos.	• Puedo elegir el rango de fechas. • El reporte incluye las propuestas más votadas y tendencias por categoría. • Puedo descargar o copiar el reporte.	Should	5	Entrega 3
HU14	Como coordinador académico, quiero ver un tablero de estadísticas, para conocer la participación de la comunidad.	• Veo totales por estado y por categoría. • Veo las 5 propuestas más votadas.	Could	3	Sustentación final
HU15	Como administrador, quiero gestionar usuarios y asignar roles, para controlar quién puede revisar propuestas.	• Puedo activar o desactivar usuarios. • Puedo asignar los roles Estudiante, Docente, Coordinador o Administrador. • Un usuario sin rol de coordinador no ve la bandeja de revisión.	Must	3	Entrega 2
HU16	Como administrador, quiero gestionar las categorías, para mantener organizado el catálogo de propuestas.	• Puedo crear, editar y eliminar categorías. • No puedo eliminar una categoría con propuestas asociadas.	Should	2	Entrega 2

ID	Historia de usuario	Criterios de aceptación	Prior.	Pts	Entrega
HU17	Como desarrollador o integrador, quiero consultar la documentación Swagger del microservicio, para entender y probar sus endpoints.	• La documentación está en /api/docs y es pública. • Cada endpoint muestra parámetros, respuestas y ejemplos. • Puedo probar los endpoints desde Swagger UI.	Must	3	Entrega 3

Total estimado: 60 puntos de historia.

12. Matriz de trazabilidad inicial

Formato definido por la guía institucional: RF → Caso de uso → Historia de usuario → Componente → Evidencia en video → Entrega → URL desplegada. Para los requisitos implementados en la Entrega 2, la columna URL indica la ruta dentro de la URL pública de Render (sección 14.5).

RF	CU	HU	Componente	Evidencia en video	Entrega	URL
RF01	CU01	HU01	usuarios	Video 2 — Registro	Entrega 2	/cuenta/registro/
RF02	CU02	HU02	usuarios	Video 2 — Login	Entrega 2	/cuenta/login/
RF03	CU03	HU03	usuarios	Video 2 — Demo funcional	Entrega 2	/cuenta/perfil/
RF04	CU15	HU15	usuarios	Video 2 — Roles	Entrega 2	/revision/
RF05	CU04	HU04	propuestas	Video 2 — CRUD	Entrega 2	/propuestas/nueva/
RF06	CU05	HU05	propuestas	Video 2 — CRUD	Entrega 2	/propuestas/<id>/editar/
RF07	CU06	HU06	propuestas	Video 2 — CRUD	Entrega 2	/propuestas/
RF08	CU07	HU07	participacion	Video 2 — Demo funcional	Entrega 2	/propuestas/<id>/votar/
RF09	CU08	HU08	participacion	Video 2 — Demo funcional	Entrega 2	/propuestas/<id>/comentar/
RF10	CU09	HU09	integraciones · ms_ia (Servicio IA)	Video 3 — Integración IA	Entrega 3	Pendiente
RF11	CU10	HU10	integraciones · ms_ia (Servicio IA)	Video 3 — Integración IA	Entrega 3	Pendiente
RF12	CU11	HU11	revision	Video 2 — Roles	Entrega 2	/revision/<id>/estado/
RF13	CU16	HU12	integraciones · ms_ia (Notificaciones)	Video 3 — Comunicación Django → Node.js	Entrega 3	Pendiente
RF14	CU12	HU13	reportes · ms_ia (Servicio IA)	Video 3 — Integración IA	Entrega 3	Pendiente
RF15	CU13	HU14	revision	Video final	Sustentación final	Pendiente
RF16	CU14	HU16	propuestas · Django Admin	Video 2 — CRUD	Entrega 2	/admin/propuestas/categoria/

RF	CU	HU	Componente	Evidencia en video	Entrega	URL
RF17	CU15	HU15	Django Admin · usuarios	Video 2 — Roles	Entrega 2	/admin/usuarios/usuario/
RF18	CU09, CU10, CU12, CU16	HU17	ms_ia (API REST · Swagger UI)	Video 3 — Swagger	Entrega 3	Pendiente

13. Repositorio y organización del proyecto

13.1 Estructura del repositorio

- **`django_app/`** — Proyecto Django «proponeu» con las apps usuarios, propuestas, participacion, revision, reportes e integraciones, una por cada módulo del diagrama de componentes.
- **`ms_ia/`** — Microservicio Node.js (se implementa en la Entrega 3).
- **`docs/`** — SRS, backlog y diagramas UML.
- **`render.yaml`** — Configuración del despliegue en Render (servicio web y base de datos).
- **`.env.example`** — Variables de entorno necesarias, sin valores secretos.
- **`README.md`** — Descripción del proyecto, arquitectura e instrucciones de ejecución.

13.2 Flujo de trabajo (Git Flow)

- **main:** versión estable desplegada en producción.
- **develop:** integración de las funcionalidades terminadas.
- **feature/HUxx-descripcion:** una rama por historia de usuario, integrada a develop mediante pull request revisado por otro integrante.
- **release/entrega-N y hotfix/*:** preparación de cada entrega y correcciones urgentes.
- **Commits:** convención Conventional Commits (feat, fix, docs, test, chore) con referencia a la historia, por ejemplo «feat(propuestas): crear propuesta (HU04)».

14. Estado de implementación — Entrega 2

Esta sección documenta el núcleo funcional construido en la Entrega 2: la aplicación Django con autenticación, roles, CRUD principal y panel administrativo, su modelo de datos, la contenerización con Docker y el despliegue inicial en Render.

14.1 Funcionalidades implementadas

RF	Funcionalidad	Ruta	Estado
RF01	Registro de estudiantes y docentes	`/cuenta/registro/`	Implementado
RF02	Inicio y cierre de sesión con correo y contraseña	`/cuenta/login/`	Implementado
RF03	Consulta y edición del perfil	`/cuenta/perfil/`	Implementado
RF04	Control de acceso por rol (403 si el rol no tiene permiso)	`/revision/`	Implementado
RF05	Crear propuesta (publicar o guardar como borrador)	`/propuestas/nueva/`	Implementado
RF06	Editar y eliminar propuesta propia en Borrador o Publicada	`/propuestas/<id>/editar/`	Implementado
RF07	Listado público con búsqueda, filtros, orden y paginación	`/propuestas/`	Implementado
RF08	Voto único a favor o en contra, modificable	`/propuestas/<id>/votar/`	Implementado
RF09	Comentarios en propuestas publicadas	`/propuestas/<id>/comentar/`	Implementado
RF12	Bandeja del coordinador y cambio de estado con observación e historial	`/revision/<id>/estado/`	Implementado
RF16	Gestión de categorías en el panel administrativo	`/admin/propuestas/categoria/`	Implementado
RF17	Gestión de usuarios y asignación de roles en el panel administrativo	`/admin/usuarios/usuario/`	Implementado
RF10, RF11, RF13, RF14, RF18	IA, propuestas similares, notificaciones, reportes y Swagger	–	Entrega 3
RF15	Tablero de estadísticas	–	Sustentación final

14.2 Modelo de datos

Los modelos se definen con el ORM de Django y se crean en la base de datos mediante migraciones. Cada aplicación del diagrama de componentes es dueña de sus modelos.

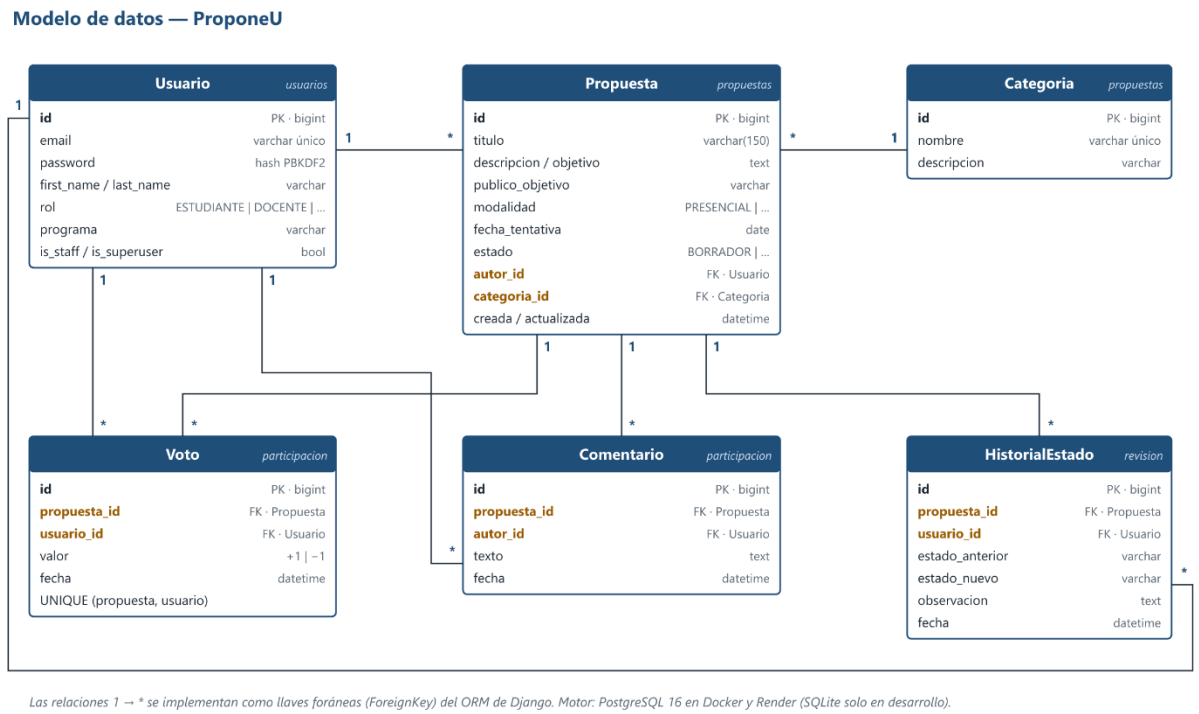


Figura 7. Modelo de datos de ProponeU.

Modelo	App	Descripción
Usuario	usuarios	Usuario personalizado (AbstractUser) que inicia sesión con correo. Guarda el rol y el programa.
Categoria	propuestas	Tipo de actividad. Se cargan seis categorías iniciales con una migración de datos.
Propuesta	propuestas	Propuesta académica con su autor, categoría, modalidad, fecha tentativa y estado.
Voto	participacion	Voto +1 o −1. Una restricción UNIQUE (propuesta, usuario) garantiza un voto por persona.
Comentario	participacion	Comentario de un usuario sobre una propuesta.
HistorialEstado	revision	Auditoría de cada cambio de estado: estado anterior, nuevo, observación, usuario y fecha (RNF11).

14.3 Roles y permisos

Acción	Visitante	Estudiante / Docente	Coordinador	Administrador
Ver y buscar propuestas	Sí	Sí	Sí	Sí
Registrarse	Sí	—	—	—
Crear, editar y eliminar propuestas propias	No	Sí	Sí	Sí
Votar y comentar	No	Sí	Sí	Sí

Acción	Visitante	Estudiante / Docente	Coordinador	Administrador
Bandeja de revisión y cambio de estado	No	No	Sí	Sí
Panel administrativo (usuarios, roles, categorías)	No	No	No	Sí

Los roles Estudiante y Docente se eligen al registrarse. Los roles Coordinador académico y Administrador solo los asigna un administrador. Las vistas protegidas usan LoginRequiredMixin y un mixin propio (CoordinadorRequeridoMixin) que responde 403 cuando el rol no tiene permiso.

14.4 Contenerización con Docker

La aplicación Django tiene su propio `Dockerfile` en `django_app/`:

- Imagen base python:3.13-slim; primero se instalan las dependencias para aprovechar la caché de capas.
- collectstatic genera los archivos estáticos, que sirve WhiteNoise sin necesidad de un servidor adicional.
- El contenedor se ejecuta con un usuario sin privilegios de administrador.
- Al arrancar, aplica las migraciones, carga opcionalmente los datos de demostración (CARGAR_DEMO) y levanta Gunicorn en el puerto definido por la variable PORT.
- La configuración se lee de variables de entorno: DJANGO_SECRET_KEY, DEBUG, DATABASE_URL, DEMO_PASSWORD.

14.5 Despliegue inicial

El despliegue se define como código en el archivo `render.yaml` (Blueprint de Render), que crea dos recursos:

- **proponeu-web:** Web Service que construye la imagen desde django_app/Dockerfile, con DEBUG=False, una clave secreta generada por Render y HTTPS.
- **proponeu-db:** PostgreSQL gestionado, conectado a la aplicación mediante la variable DATABASE_URL.

URL pública: [completar con la URL asignada por Render]

14.6 Pruebas automatizadas

Se implementaron 29 pruebas con el framework de pruebas de Django, que se ejecutan con python manage.py test:

App	Qué se prueba
usuarios	Registro, correo único, rol no permitido en el registro, login, logout y permisos por rol.
propuestas	CRUD completo, login obligatorio, permisos del autor, estados no editables, búsqueda, filtros y privacidad de los borradores.
participacion	Voto único y modificable, no votar la propia propuesta, votación solo en publicadas y comentarios.

App	Qué se prueba
revision	Acceso exclusivo del coordinador, historial al aprobar y observación obligatoria al rechazar.